

Bhai, "**First Negative Integer in Every Window of Size K**" bhi ek bohot hi mast Sliding Window category ka question hai. Interviewer jab iska **Brute Force Approach** poochta hai, toh wo dekhna chahta hai ki tum constraints ko bina optimize kiye basic loops se kaise fold karte ho.

Chalo iska poora logic, dry-run aur code ekdum simple bhasha me samajhte hain.

1. Brute Force Ka Logic (Nested Loops)

Maan lo hamare paas ek array hai aur hume K size ki har ek window me sabse pehla negative number dhoondna hai. Agar kisi window me koi negative number nahi hai, toh hume 0 rakhna hai.

Brute Force me hum simple **Nested Loops** ka use karte hain:

1. **Outer Loop (i):** Yeh loop har ek window ki starting position ko fix karega. Agar array ka size N hai aur window size K hai, toh total windows $N - K + 1$ banengi. Isliye outer loop 0 se lekar $N - K$ tak chalega.
 2. **Inner Loop (j):** Yeh loop current window ke andar ghumega, yaani i se lekar $i + K - 1$ tak.
 3. Inner loop ke andar hum check karenge ki kya koi element < 0 (negative) hai?
- Jaise hi pehla negative element milega, hum use apne result me save karenge aur **break** karke agli window par chale jayenge (kyunki hume sirf **first** negative chahiye).
 - Agar poori window ghumne ke baad bhi koi negative nahi mila, toh hum result me 0 daal denge.

2. Brute Force C++ Code ($O(N \times K)$)

```
#include <iostream>
#include <vector>
using namespace std;

class Solution {
public:
    vector<int> firstNegativeBrute(vector<int>& arr, int k) {
        int n = arr.size();
        vector<int> result;

        // Outer loop: Har window ki shuruat track karne ke liye
        for (int i = 0; i <= n - k; i++) {
            bool flag = false; // Track karega ki negative mila ya nahi

            // Inner loop: Current window ke andar check karne ke liye (size K)
            for (int j = i; j < i + k; j++) {
                if (arr[j] < 0) {
                    result.push_back(arr[j]); // Pehla negative mil gaya, save
                    flag = true;
                    break; // Aage check karne ki zaroorat nahi, break the inner
                }
            }

            if (!flag) result.push_back(0);
        }
    }
};
```

```

    }

    // Agar poori window me koi negative nahi mila, toh 0 daalo
    if (!flag) {
        result.push_back(0);
    }
}

return result;
}
};

int main() {
    vector<int> arr = {12, -1, -7, 8, -15, 30, 16, 28};
    int k = 3;

    Solution sol;
    vector<int> ans = sol.firstNegativeBrute(arr, k);

    cout << "Brute Force Result: ";
    for (int x : ans) {
        cout << x << " ";
    }
    cout << endl;
    // Output: -1 -1 -7 -15 -15 0

    return 0;
}

```

3. Complexity Analysis (Brute Force Kyun Slow Hai?)

1. Time Complexity: $O(N \times K)$

- **Kyun?** Kyunki total windows lagbhag N banti hain, aur har window ke andar hum maximum K elements ko check kar rahe hain.
- Agar array ka size 10^5 ho aur window size $K = 10^4$ ho, toh total operations 10^9 ke paas pahunch jayenge, jo platform par **TLE (Time Limit Exceeded)** de dega.

2. Space Complexity: $O(1)$ Auxiliary Space

- **Kyun?** Jo `result` vector humne banaya hai wo toh answer return karne ke liye mandatory hai. Uske alawa humne koi extra temporary space (jaise Queue ya Map) use nahi kiya, sirf loop variables use kiye hain.

4. Edge Cases Par Iska Post-Mortem

Brute force slow hai, par ye in edge cases par safely kaam karta hai:

1. **Array Me Ek Bhi Negative Na Ho** ($[1, 2, 3, 4], K = 2$): Inner loop chalega, negative nahi milega, `flag` false rahega aur har window ke liye `0` push hota chalega. Output: $[0, 0, 0]$. **(PASS ✓)**
2. **Saare Hi Elements Negative Hon** ($[-1, -2, -3], K = 2$): Har window ke inner loop me pehle hi index (`j = i`) par condition hit ho jayegi, `break` chalega aur turant output milega. **(PASS ✓)**
3. **Window Size Array Ke Barabar Ho** ($K = N$): Outer loop sirf ek baar chalega (`i = 0` se `0`). Inner loop poore array ko scan karke pehla negative return karega. **(PASS ✓)**
4. **Duplicates Ya Zeros Present Hon**: Pointers index-by-index aage badhte hain, toh values zero hon ya repeat ho rahi hon, conditional statement (`arr[j] < 0`) unhe perfectly handle karega. **(PASS ✓)**

Bhai, brute force ekdum sahi se samajh aa gaya? Ab is $O(N \times K)$ ko tod kar Sliding Window technique se $O(N)$ **Optimal Approach** (using Deque or Queue) me kaise badalte hain, wo bataun?

Bhai, is question ke liye **Better Approach** ka matlab hai ki hum brute force ke $O(N \times K)$ time ko optimize karke $O(N)$ **Time** par le aayein, lekin uske liye hum thoda sa extra space use karein (jaise ek standard **Queue**).

Brute force me dikkat kya thi? Hum har window me baar-baar piche jaakar elements ko dhoond rahe the. Better approach me hum ek hi baar array par aage badhenge aur jo bhi negative numbers milenge, unka track rakh lenge.

1. Better Logic: Queue Ka Use ($O(N)$ Time, $O(N)$ Space)

Hum ek standard `queue<int>` banayenge jo sirf aur sirf **negative numbers ke indices (ya values)** ko store karegi.

Khel Kaise Chalta Hai? (Two Steps)

Step 1: Pehli Window Ko Process Karo (Pehle K elements)

- Loop chalao `0` se `K-1` tak. Agar koi element negative (`< 0`) milta hai, toh use queue me push kar do.
- Pehli window ka answer kya hoga? Agar queue khaali nahi hai, toh queue ka sabse aage wala element (`q.front()`) hi hamara pehla negative hai. Agar queue khaali hai, toh answer `0` hai.

Step 2: Sliding Window Chalao (Baaki saare elements ke liye) Ab hum window ko ek-ek step aage badhayenge (Index `i` chalega `K` se `N-1` tak):

1. **Purane element ko nikaalo**: Jo element ab window se bahar ja raha hai (yaani index `i - K` wala element), check karo kya wo queue ke front me baitha hai? Agar haan, toh use queue se hata do (`q.pop()`).
2. **Naye element ko add karo**: Jo naya element window me ghus raha hai (`arr[i]`), agar wo negative hai, toh use queue me daal do.
3. **Answer Note Karo**: Is naye window ka answer fir se wahi hoga—agar queue empty nahi hai toh `q.front()`, nahi toh `0`.

2. Better C++ Code ($O(N)$ Time, $O(N)$ Space)

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Solution {
public:
    vector<int> firstNegativeBetter(vector<int>& arr, int k) {
        int n = arr.size();
        queue<int> q; // Sirf negative numbers ko hold karne ke liye
        vector<int> result;

        // Step 1: Pehli window (K size) ke negative numbers queue me daalo
        for (int i = 0; i < k; i++) {
            if (arr[i] < 0) {
                q.push(arr[i]); // Hum direct values store kar rahe hain
            }
        }

        // Pehli window ka result store karo
        if (!q.empty()) result.push_back(q.front());
        else result.push_back(0);

        // Step 2: Window ko slide karo i = K se n-1 tak
        for (int i = k; i < n; i++) {

            // Outgoing element check: Kya window se bahar jaane wala element
            // queue me tha?
            // arr[i - k] wahi element hai jo ab is window ka part nahi raha
            if (arr[i - k] < 0 && !q.empty() && q.front() == arr[i - k]) {
                q.pop();
            }

            // Incoming element check: Kya naya element negative hai?
            if (arr[i] < 0) {
                q.push(arr[i]);
            }

            // Current window ka answer nikal lo
            if (!q.empty()) result.push_back(q.front());
            else result.push_back(0);
        }

        return result;
    }
};

int main() {
    vector<int> arr = {12, -1, -7, 8, -15, 30, 16, 28};
    int k = 3;

    Solution sol;
```

```

vector<int> ans = sol.firstNegativeBetter(arr, k);

cout << "Better Approach Result: ";
for (int x : ans) {
    cout << x << " ";
}
cout << endl;
// Output: -1 -1 -7 -15 -15 0

return 0;
}

```

3. Complexity Analysis

- **Time Complexity:** $O(N)$ 🔥
- **Kyun?** Kyunki humne pure array par sirf ek hi baar travel kiya hai. Ek loop pehli window ke liye chala (K times) aur dusra loop bache hue elements ke liye chala ($N - K$ times). Total operations strictly N hain, isliye nested loops ka jhanjhat khatam!
- **Space Complexity:** $O(N)$ ❌
- **Kyun?** Worst case me agar array ke saare elements negative hon (jaise $[-1, -2, -3, -4]$), toh hamari queue me saare ke saare elements push ho jayenge. Is wajah se extra space $O(N)$ lag sakti hai.

4. Edge Cases Par Test

1. **Koi Negative Na Ho** ($[1, 2, 3]$): Queue hamesha empty rahegi, code har baar `else result.push_back(0)` me jayega aur safely saare windows ke liye `0` de dega. **(PASS ✅)**
2. **Saare Hi Negative Hon** ($[-5, -2, -3]$): Queue me saare elements bharenge, pop aur push unke serial sequence ke mutabik sahi chalega aur har window ka front perfect aayega. **(PASS ✅)**
3. **Duplicates Elements** ($[-1, -1, 2]$): Kyunki hum direct values compare kar rahe hain `q.front() == arr[i - k]`, yahan ek chota sa catch hai—agar duplicates windows me cross-over karenge toh values me mismatch ho sakta hai.

Interviewer Trap Alert 🚨: Interviewer yahan tumhe pakad sakta hai: "Bhai, agar duplicate values hain, toh direct value store karne se `pop` galat element ko kar sakta hai. Isse bachne ke liye hame indices store karne chahiye!" > Aur yahi se janam hota hai **Optimal Approach** ka, jahan hum values ke bajay **Indices** store karte hain ek `deque` me, jisse Space bhi control ho jati hai max $O(K)$ aur duplicates ka lafda bhi khatam!

Bhai, samajh aaya ki Queue se time $O(N)$ toh ho gaya, par duplicate elements par ye phans sakta hai? Isko ekdum makkhan karne ke liye **Optimal Approach** ($O(N)$ Time & $O(K)$ Space) dekhein?

Bhai, ab aate hain is question ke **Absolute Optimal Approach** par.

Pichli approach me jo duplicate vaala flaw tha (jahan values mismatch ho sakti thi) aur jo space worst-case me $O(N)$ ja rahi thi, un dono problems ko hum yahan khatam kar denge. Isme hum values ki jagah **Indices** ka track rakhenge aur **queue** ki jagah **deque (Double Ended Queue)** use karenge.

Isse space strictly limits me rahegi—**Maximum $O(K)$** (kyuki ek window me K se zyada elements ho hi nahi sakte) aur Time hamesha $O(N)$ rahega.

1. Optimal Logic: Indices + Deque Ka Khel

Hum ek `deque<int> dq` banayenge jo sirf un elements ke **indices** store karega jo negative hain.

Steps Kaise Kaam Karenge?

Step 1: Pehli Window (Pehle K elements)

- 0 se $K-1$ tak loop chalao. Agar `arr[i] < 0` hai, toh uska index `i` deque ke piche push kar do (`dq.push_back(i)`).
- Pehli window ka answer: Agar deque khaali nahi hai, toh `arr[dq.front()]` hamara pehla negative hai. Agar khaali hai, toh `0`.

Step 2: Window Ko Slide Karo ($i = K$ se $N-1$ tak) Har naye element par do cheezein check karni hain:

1. **Out of Bound Indices Ko Hatao:** Jo index ab current window se bahar ho chuka hai (yani agar `dq.front() == i - k`), use aage se nikal do (`dq.pop_front()`). Yeh step hamesha track rakhta hai ki deque me sirf current window ke hi elements bachein.
2. **Naya Element Add Karo:** Agar naya incoming element `arr[i] < 0` hai, toh uska index `i` piche se push kar do (`dq.push_back(i)`).
3. **Answer Note Karo:** Agar deque khaali nahi hai, toh `arr[dq.front()]` push karo, nahi toh `0`.

2. Optimal C++ Code ($O(N)$ Time, $O(K)$ Space)

```
#include <iostream>
#include <vector>
#include <deque>
using namespace std;

class Solution {
public:
    vector<int> firstNegativeOptimal(vector<int>& arr, int k) {
        int n = arr.size();
        deque<int> dq; // Yeh negative elements ke INDICES store karega
        vector<int> result;

        // Step 1: Pehli window ke negative elements ke indices nikalo
        for (int i = 0; i < k; i++) {
            if (arr[i] < 0) {
                dq.push_back(i);
            }
        }
    }
};
```

```

    }

    // Pehli window ka result note karo
    if (!dq.empty()) result.push_back(arr[dq.front()]);
    else result.push_back(0);

    // Step 2: Baaki windows ke liye slide karo
    for (int i = k; i < n; i++) {

        // 1. Outgoing check: Jo index window ke bahar chala gaya use hatao
        if (!dq.empty() && dq.front() == i - k) {
            dq.pop_front();
        }

        // 2. Incoming check: Agar naya element negative hai toh index add
        karo
        if (arr[i] < 0) {
            dq.push_back(i);
        }

        // 3. Current window ka result note karo
        if (!dq.empty()) result.push_back(arr[dq.front()]);
        else result.push_back(0);
    }

    return result;
}
};

int main() {
    vector<int> arr = {12, -1, -1, 8, -15, 30, 16, 28};
    int k = 3;

    Solution sol;
    vector<int> ans = sol.firstNegativeOptimal(arr, k);

    cout << "Optimal Approach Result: ";
    for (int x : ans) {
        cout << x << " ";
    }
    cout << endl;
    // Output: -1 -1 -1 -15 -15 0

    return 0;
}

```

3. Complexity Analysis (Interviewer Selection Point)

- **Time Complexity:** $O(N)$ 🔥

- **Kyun?** Hum array par sirf ek baar linear scan kar rahe hain. Deque me har ek index maximum ek hi baar push hota hai aur ek hi baar pop hota hai, jisse saare operations constant time $O(1)$ me hote hain.
- **Space Complexity:** $O(K)$ 🔥
- **Kyun?** Deque me hum maximum utne hi indices store kar sakte hain jo ek window ke andar aate hain. Window ka size K hai, toh worst case me bhi deque ka size kabhi bhi K se bada nahi ho sakta. Yeh $O(N)$ se bohot behtar hai agar $K \ll N$ ho.

4. Saare Edge Cases Ka Dry-Run

1. Duplicate Elements (`[-1, -1, 2]`, $K = 2$):

- Pehli window me `dq` me indices `[0, 1]` jayenge. Jab window slide hogi, toh index `0` bahar jayega, `dq` me `[1]` bachega. Kyunki hum index compare kar rahe hain (`dq.front() == i - k`), values same hone par bhi data mix-up ka koi chance nahi hai! perfectly **PASS** ✅

2. Array Me Koi Negative Na Ho (`[5, 10, 15]`):

- Deque poore loop me ekdum khaali (`empty`) rahegi. Code hamesha `else result.push_back(0)` me jayega aur safely `0 0...` return kar dega. (**PASS** ✅)

3. Saare Ke Saare Negative Hon (`[-2, -3, -4, -5]`):

- Deque me continuous indices add hote rahenge aur front se purane valid indices slip out hote rahenge. Window strict limits me perfect answer degi. (**PASS** ✅)

4. K Array Size Ke Barabar Ho ($K = N$):

- Pehla loop `0` se `N-1` tak chalega aur poore array ka pehla negative dhoond lega. Sliding waala second loop chalega hi nahi, safely correct execution hoga. (**PASS** ✅)

Bhai, ab Sliding Window ka ye concept, duplicate elements ko index se handle karne ki trick, aur $O(K)$ ki space management ekdum solid samajh aa gayi?

Bhai, "**First Negative Integer in Every Window of Size K**" ke is Optimal Deque Approach par interviewer jab edge cases fekta hai, toh wo yeh dekhna chahta hai ki tumhara code boundary boundaries par fatega toh nahi, aur kya tum `i - k` ka logic sahi se deploy kar rahe ho.

Chalo, is optimal indices-tracking logic ke **saare critical edge cases** ka solid dry-run dekhte hain taaki interview me tumhara confidence ekdum heavy rahe.

Hamara optimal check kya hai:

1. Outgoing element hatana: `if (!dq.empty() && dq.front() == i - k) dq.pop_front();`
2. Incoming element jodna: `if (arr[i] < 0) dq.push_back(i);`

1. Window Size Jab Array Ke Barabar Ho ($K = N$)

Situation: Maan lo array hai `arr = [10, -2, -3]`, aur window size $K = 3$ hai.

Dry Run:

- `targetTickets` ya array size $N = 3$. Pehla loop $i = 0$ se 2 tak poora chal jayega.
 - $i = 0$ (10): Kuch nahi hoga.
 - $i = 1$ (-2): `dq` me index 1 push ho gaya. `dq = [1]`
 - $i = 2$ (-3): `dq` me index 2 push ho gaya. `dq = [1, 2]`
 - **Pehli window ka result:** `dq.front()` par index 1 hai, toh `arr[1]` yaani -2 result me chala gaya.
 - **Sliding Loop ($i = K$ se $N-1$):** $i = 3$ par loop condition ($3 < 3$) false ho jayegi. Loop chalega hi nahi!
 - **Result:** Final answer sirf $[-2]$ aayega, jo ki bilkul sahi hai. Code bina kisi extra run ke safely **PASS**  ho gaya.
-

2. Duplicate Negative Elements Ka Crossover (`arr = [-5, -5, 2, 4]`, $K = 2$)

Situation: Pehli approach me value store karne par duplicates aapas me phas rahe the. Chalo dekhte hain index-tracking se kaise bachenge.

Dry Run:

- **Step 1 (Pehli Window $i=0, 1$):**
 - $i = 0$ (-5): `dq.push_back(0)` $\rightarrow$ `dq = [0]`
 - $i = 1$ (-5): `dq.push_back(1)` $\rightarrow$ `dq = [0, 1]`
 - Result stored: `arr[dq.front()] = arr[0] = -5`.
 - **Step 2 (Sliding Window $i = 2$, Value = 2):**
 - **Outgoing Check:** $i - k = 2 - 2 = 0$. Kya `dq.front() == 0`? Haan! Toh index 0 pop ho gaya. Ab `dq = [1]`.
 - **Incoming Check:** `arr[2]` (2) negative nahi hai, toh kuch push nahi hoga.
 - Result stored: `arr[dq.front()] = arr[1] = -5`.
 - **Result:** Dono windows ke liye perfect -5 aur -5 mila. Chunki humne values ke bajay unique memory positions (indices 0 aur 1) ka track rakha, isliye duplicates hamare logic ko fool nahi kar paaye! (**PASS** )
-

3. Window Me Ek Bhi Negative Na Ho (`arr = [10, 20, 30, -5]`, $K = 2$)

Situation: Shuruat ki windows me saare positive numbers hain, aur aakhiri me ek negative chupa hai.

Dry Run:

- **Pehli Window ($i=0, 1$):** 10 aur 20 dono positive hain. `dq` ekdum khaali (`[]`) rahegi.
- Result stored: `dq.empty()` hai, toh 0 push hua.

- **Sliding Step 1 ($i = 2$, Value = 30):**
 - Outgoing check bypass (dq empty hai). Incoming 30 positive hai. dq abhi bhi [].
 - Result stored: 0.
 - **Sliding Step 2 ($i = 3$, Value = -5):**
 - Outgoing check bypass. Incoming -5 negative hai, toh `dq.push_back(3)`. dq = [3].
 - Result stored: `arr[3] = -5`.
 - **Result:** Output aaya [0, 0, -5]. Jahan negative nahi tha, wahan bina kisi garbage value ke perfectly 0 maintain raha. (PASS )
-

4. Saare Ke Saare Elements Negative Hon (`arr = [-1, -2, -3]`, $K = 2$)

Situation: Array me koi positive number hai hi nahi. Deque ka size kaise control hoga?

Dry Run:

- **Pehli Window ($i=0, 1$):** dq me indices [0, 1] chale gaye. Result = `arr[0] = -1`.
 - **Sliding Step ($i = 2$, Value = -3):**
 - **Outgoing Check:** $i - k = 2 - 2 = 0$. `dq.front()` par 0 baitha hai, toh use aage se pop kiya (`dq.pop_front()`). Ab dq = [1].
 - **Incoming Check:** $-3 < 0$ hai, toh index 2 piche se push hua. Ab dq = [1, 2].
 - Result stored: `arr[1] = -2`.
 - **Result:** Deque ka size hamesha strictly under limit (K) raha aur output perfectly [-1, -2] aa gaya. (PASS )
-

5. Window Size Jab 1 Ho ($K = 1$)

Interviewer Special Twist: "Bhai, agar window ka size sirf 1 hi ho, toh tumhara sliding window logic chalega?"

Dry Run: Maan lo `arr = [10, -5, 2]`, $K = 1$

- **Pehli Window ($i = 0$):** 10 positive hai, dq = []. Result = 0.
- **Sliding Step 1 ($i = 1$, Value = -5):**
- Outgoing check: $i - k = 1 - 1 = 0$. dq empty hai, no pop.
- Incoming check: $-5 < 0$ hai, `dq.push_back(1)`. dq = [1].
- Result stored: `arr[1] = -5`.

- **Sliding Step 2 ($i = 2$, Value = 2):**
- Outgoing check: $i - k = 2 - 1 = 1$. `dq.front() == 1` hai! Toh index 1 pop ho gaya. `dq = []`.
- Incoming check: 2 positive hai, no push.
- Result stored: 0.
- **Result:** Output आया `[0, -5, 0]`. Single element windows par bhi array bounds ya subtraction logic ekdum makkhan chala. (PASS )

Edge Case Checklist for Interview

Edge Case Scenario	Potential Danger	Why Optimal Deque Wins?
$K = N$ (Max Window)	Loop out of bounds or index error	Second loop automatically bypass ho jata hai because of condition $i < n$.
Duplicate Elements	Erroneous pops / wrong value tracking	Values ke bajay indices compare hote hain (<code>dq.front() == i - k</code>), error zero.
No Negatives Present	Garbage vector insertion or crashes	<code>dq.empty()</code> structural safety lagayi hai, strictly 0 insert hota hai.
$K = 1$ (Min Window)	Outgoing logic mismatch	$i - k$ calculation mathematically 1-size check ko point-to-point handle karti hai.

Bhai, ab sliding window ke is super important question ke saare brute, better, optimal, aur edge cases par tumhari aisi command ho gayi hai ki kisi bhi interview me tum iska poora structure khol kar rakh sakte ho!

Makkhan clear hai ab poora khel?

Print All Number in Every Window of Size K

Bhai, "Print All Numbers in Every Window of Size K" ka **Brute Force Approach** ekdum seedha aur sasta tarika hai. Iska simple funda hai: "Har ek window ki starting position par jaakar khade ho jao, aur wahan se line se K elements ko print kar do."

Chalo iska poora dry-run aur inner logic bilkul basic se samajhte hain.

1. Brute Force Ka Logic (Nested Loops)

Maan lo hamare paas ek array hai aur hume K size ki har ek window ke saare elements ko print karna hai. Hum do loops ka use karenge:

1. **Outer Loop (i):** Yeh loop tay karega ki hamari current window **shuru** kahan se ho rahi hai.
- Agar array ka size N hai aur window size K hai, toh aakhiri window index $N - K$ se shuru ho sakti hai (kyuki uske baad hi K elements bachenge). Isiliye outer loop 0 se $N - K$ tak chalega.

2. **Inner Loop (j):** Yeh loop current window ke andar ghumega.

- Yeh i se shuru hoga (jahan se window shuru hui) aur exactly K elements aage tak jayega ($i + K$). Is loop ke andar hum ek-ek karke saare elements ko screen par print karte jayenge.

2. Brute Force C++ Code ($O(N \times K)$)

```
#include <iostream>
#include <vector>
using namespace std;

class Solution {
public:
    void printWindowsBrute(vector<int>& arr, int k) {
        int n = arr.size();

        // Outer loop: Har window ki starting position fix karega (0 se n-k)
        for (int i = 0; i <= n - k; i++) {
            cout << "Window " << i + 1 << " -> [ ";

            // Inner loop: Current window ke saare K elements ko print karega
            for (int j = i; j < i + k; j++) {
                cout << arr[j] << " ";
            }
            cout << "]" << endl;
        }
    }
};

int main() {
    vector<int> arr = {10, 20, 30, 40, 50};
    int k = 3;

    Solution sol;
    sol.printWindowsBrute(arr, k);

    return 0;
}

/* Output:
Window 1 -> [ 10 20 30 ]
Window 2 -> [ 20 30 40 ]
Window 3 -> [ 30 40 50 ]
*/
```

3. Complexity Analysis

- Time Complexity:** $O(N \times K)$ ❌

- **Kyun?** Kyuki total windows lagbhag N banti hain ($N - K + 1$), aur har window ke andar hamara inner loop strictly K baar chalta hai elements ko print karne ke liye.
- Agar array bada ho ($N = 10^5$) aur window size bhi bada ho ($K = 10^4$), toh total iterations 10^9 ho jayengi, jo system ko slow kar dengi.
- **Space Complexity:** $O(1)$ 🔥
- **Kyun?** Kyuki humne koi extra data structure (jaise queue, vector ya map) use nahi kiya data store karne ke liye. Hum seedha terminal par array se utha kar elements print kar rahe hain.

4. Saare Edge Cases Par Iska Dry-Run

1. Window Size Array Ke Barabar Ho ($K = N$):

- Maan lo `arr = [1, 2, 3]` aur `k = 3`.
- Outer loop sirf ek baar chalega (`i = 0` se `0 <= 0`). Inner loop `j = 0` se `3` tak chalega aur poora array ek baar me print kar dega: `[ 1 2 3 ]`. **(PASS ✓)**

2. Window Size Exactly 1 Ho ($K = 1$):

- Maan lo `arr = [10, 20, 30]` aur `k = 1`.
- Outer loop `0` se `2` tak chalega (`i <= 3 - 1`). Inner loop har baar sirf ek hi element print karega (`j = i` se `j < i + 1`).
- Output: `[ 10 ], [ 20 ], [ 30 ]`. Ekdum perfect. **(PASS ✓)**

3. Negative aur Duplicate Numbers:

- Loops ko numbers ke signs ya unki values se koi matlab nahi hota, wo strictly indices ke coordinates par kaam karte hain. Isiliye array me duplicate ho ya zeroes, ye bina ruke sabko sahi order me print karega. **(PASS ✓)**

Bhai, brute force ka ye simple nested loop wala funda ekdum sahi se dimaag me set hua? Ab iska better/optimal tarika dekhna hai?

Bhai, ab aate hain is question ke **Better Approach** par.

Brute force me dikkat kya thi? Jab hum ek window se agli window par shift ho rahe the, toh hum beech wale saare elements ko dobara print karne ke liye loop chala rahe the. Agar array bada ho, toh ye redundant operations system ko slow kar dete hain.

Better Approach ka logic ye hai: Hum **Two Pointers** ka use karke ek actual window frame banayenge aur bina kisi nested loop ke poore array par sirf **ek baar linear scan** karenge. Jab tak hamari window पूरी nahi hoti, hum aage badhte rahenge, aur jaise hi window ka size K ho jayega, hum use print karke aage slide kar denge.

1. Better Logic: Two Pointers (Sliding Window)

Hum do pointers maintain karenge:

- **start**: Window ka shuruat ka point track karne ke liye.
- **end**: Window ka aakhiri point track karne ke liye.

Steps Kaise Kaam Karenge?

1. Shuruat me **start** = 0 aur **end** = 0 rakho.
2. **end** pointer ko ek-ek karke aage badhao jab tak wo array ke khatam hone tak nahi pahunchta.
3. **Check**: Kya hamari current window ka size K ke barabar ho gaya hai? Window ka size nikalne ka formula hota hai: $(\text{end} - \text{start} + 1)$.
4. Jaise hi window size K ke barabar ho jaye:
 - **start** se lekar **end** tak ke elements ko print kar do.
 - Agli window par jaane ke liye **start** pointer ko ek step aage badha do (**start++**).

2. Better C++ Code ($O(N \times K)$ Processing, $O(1)$ Auxiliary Space)

```
#include <iostream>
#include <vector>
using namespace std;

class Solution {
public:
    void printWindowsBetter(vector<int>& arr, int k) {
        int n = arr.size();
        int start = 0;

        // end pointer array ke har ek element par aage badhega
        for (int end = 0; end < n; end++) {

            // Jab tak window ka size K nahi hota, tab tak loop sirf end ko
            badhayega

            if (end - start + 1 == k) {
                cout << "Window " << start + 1 << " -> [ ";

                // Current window ke elements ko print karo
                for (int i = start; i <= end; i++) {
                    cout << arr[i] << " ";
                }
                cout << "]" << endl;

                // Window ko aage slide karne ke liye start ko aage badhao
                start++;
            }
        }
    }
};

int main() {
    vector<int> arr = {10, 20, 30, 40, 50};
    int k = 3;
```

```

Solution sol;
sol.printWindowsBetter(arr, k);

return 0;
}
/* Output:
Window 1 -> [ 10 20 30 ]
Window 2 -> [ 20 30 40 ]
Window 3 -> [ 30 40 50 ]
*/

```

3. Complexity Analysis

- **Time Complexity:** $O(N \times K)$ * **Kyun?** Structural level par humne nested loops ko khatam kar diya hai aur single-pass format banaya hai. Lekin, chunki hume har ek window ke saare elements ko screen par **print** karna hi padta hai, toh wo printing loop har valid window par K baar chalega hi chalega. Isliye time complexity $O(N \times K)$ hi rahegi, kyunki output format hi demand kar raha hai.
- **Space Complexity:** $O(1)$ 🔥
- **Kyun?** Brute force ki tarah hi, humne yahan bhi koi extra intermediate space (jaise queue ya temporary vectors) use nahi kiya hai. Hum seedha pointers ka use karke screen par logging kar rahe hain.

4. Edge Cases Par Iska Test

1. Window Size Array Ke Barabar ($K = N$):

- **end** pointer badhte-badhte jab aakhiri index ($n-1$) par aayega, tabhi $end - start + 1 == k$ true hoga. Loop ke andar poora array ek hi baar me print ho jayega aur **start++** hone ke baad code end ho jayega. **(PASS ✅)**

2. Window Size Exactly 1 ($K = 1$):

- Har ek index par $end - start + 1 == k$ hamesha true hoga ($0-0+1 = 1$). Har ek single element apni khud ki ek window banakar perfectly line se print hota chalega. **(PASS ✅)**

3. Zeros aur Negative Elements:

- Yeh approach poori tarah se index limits ke calculation par banti hai, isliye elements ke andar kya values hain (positive, negative, zero, duplicate), usse logic par koi farq nahi padta. **(PASS ✅)**

Bhai, Two Pointers ka use karke sliding window frame kaise banta hai, ye clear hua? Is pure window system ko actual stack/queue structures me store karne wala optimal dimaag dekhna hai ab?

Bhai, ab aate hain is question ke **Sabse Optimal aur Standard Sliding Window Approach** par.

Pichli approach (Better Approach) me code toh sahi chal raha tha, par agar interviewer tumse bole: "Bhai, mujhe elements ko sirf terminal par print nahi karna, balki mujhe har window ke saare numbers ko ek data

structure (jaise `vector<vector<int>>`) me store karke return karna hai," toh wahan par tumhara pointers wala logic thoda phansega ya extra loops lega.

Optimal approach me hum bina kisi nested ya extra loops ke, array par sirf **ek baar linear traversal ($O(N)$)** karte hain aur **Monotonic Deque / Standard Queue / Vector Store** ka use karke window ke state ko live maintain karte hain.

1. Optimal Logic: Live Window Tracking

Hum ek `vector<int>` `currentWindow` maintain karenge jo hamari chalti-firti window ka live frame hogi:

1. **Shuruat ke K elements:** Pehle K elements ko chupchap is window me daal do (`push_back`). Jab $i = K-1$ par pahunche, toh hamari pehli window taiyar hai, use store/print kar lo.
2. **Window ko slide karna ($i = K$ se $N - 1$ तक):** Now, jab hum aage badhenge:
 - **Outgoing Element:** Window ka sabse pehla element ab purana ho chuka hai, use aage se hata do. (Agar vector use kar rahe hain, toh hum elements ko swap/shift karne ke bajay direct index jump kar sakte hain, ya fir `deque` use karke $O(1)$ me `pop_front()` kar sakte hain).
 - **Incoming Element:** Naye element ko piche se window me jod do.
 - Har ek step par hamari window hamesha updated rahegi aur hum use direct utilize kar sakte hain.

2. Optimal C++ Code ($O(N)$ Window State Management)

Interview standard ke mutabik, hum saari windows ko ek 2D vector (`vector<vector<int>>`) me store karke return karenge, jo ki absolute optimal maana jata hai:

```
#include <iostream>
#include <vector>
using namespace std;

class Solution {
public:
    vector<vector<int>> getAllWindowsOptimal(vector<int>& arr, int k) {
        int n = arr.size();
        vector<vector<int>> allWindows;
        vector<int> currentWindow;

        // Step 1: Pehli window ke K elements ko bhar lo
        for (int i = 0; i < k; i++) {
            currentWindow.push_back(arr[i]);
        }
        // Pehli window ko final list me daal do
        allWindows.push_back(currentWindow);

        // Step 2: Window ko slide karo i = K se lekar n-1 tak
        for (int i = k; i < n; i++) {

            // Palti maarne ya shift karne ke bajay, hum naye vector me
```

```

        // purane elements ko copy karne ke bajay smart window shift dikha
        sakte hain.
        // Par sabse efficient tarika bina extra structure ke sirf loops ka
        smart use hai:

        vector<int> nextWindow;
        // Hum direct array ke indices se window recreate kar rahe hain bina
        nested traversal ke
        // Kyuki sliding window me har element strictly limited move hota hai
        for(int j = i - k + 1; j <= i; j++) {
            nextWindow.push_back(arr[j]);
        }
        allWindows.push_back(nextWindow);
    }

    return allWindows;
}

};

int main() {
    vector<int> arr = {10, 20, 30, 40, 50};
    int k = 3;

    Solution sol;
    vector<vector<int>> ans = sol.getAllWindowsOptimal(arr, k);

    // Final result print karne ke liye
    for (int i = 0; i < ans.size(); i++) {
        cout << "Window " << i + 1 << " -> [ ";
        for (int x : ans[i]) {
            cout << x << " ";
        }
        cout << "]" << endl;
    }

    return 0;
}

```

3. Complexity Analysis

- **Time Complexity:** * Agar hum sirf **Window State** maintain karne ki baat karein (jaise elements ko sliding frame me dhalna), toh logic $O(N)$ me complete ho jata hai.
- Lekin, chunki is question ka core nature hi hai "saare elements ko dhoondna/print karna", toh final output list taiyar karne me overall time $O(N \times K)$ hi jayega, kyuki hum K elements ko har window me touch kar rahe hain. Duniya ka koi bhi algorithm isko isse fast nahi kar sakta kyunki output ka size hi $N \times K$ digits ka hota hai.
- **Space Complexity:** $O(K)$ **Auxiliary Space** 🔥

- **Kyun?** Ek samay par hamari `currentWindow` me strictly maximum K elements hi store hote hain. (Hum final storage wale 2D vector ko space complexity me count nahi karte kyunki wo hamara answer return karne ke liye mandatory hai).

4. Boundary Edge Cases Verification

1. Array Size aur Window Size Barabar ($K = N$):

- Pehla loop 0 se $N-1$ tak chalega aur poora array `currentWindow` me aa jayega. `allWindows.push_back` chalega.
- Second loop ($i = k$ yaani $n < n$) fail ho jayega. Output me strictly ek hi window milegi. **(PASS ✓)**

2. Window Size Ekdam Minimum ($K = 1$):

- Pehli window me sirf `arr[0]` jayega. Uske baad sliding loop $i = 1$ se $n-1$ tak chalega aur har ek single element ko apni alag window banakar 2D vector me push karta jayega. Ekdam perfect structure. **(PASS ✓)**

3. Negative, Zeros aur Duplicates:

- Yeh mathematical traversal sirf array indices ke math ($i - k + 1$) par nirbhar hai. Isliye array me chahe -1 ho, 0 ho ya bade duplicate numbers hon, elements hamesha sahi frame me hi capture honge. **(PASS ✓)**

Bhai, ab Sliding Window ke is fundamental block ke teeno patterns (Brute, Better, Optimal) aur unke micro-level adjustments poori tarah dimaag me fit baith gaye?

Bhai, jab interviewer tumse kahe ki "**Mujhe Queue ka use karke hi har window ke elements ko live track aur print karke dikhao**", toh iska matlab wo tumse asli **Sliding Window State Management** karwana chahta hai.

Queue ka sasta aur sabse bada fayda ye hai ki iska niyam hota hai **FIFO** (First In First Out). Sliding window me bhi yahi hota hai: jab window aage badhti hai, toh jo element sabse pehle aaya tha, wahi sabse pehle bahar jata hai!

Chalo iska ek-ek step ka visual dry-run aur makkhan code dekhte hain.

1. Queue Se Sliding Window Ka Logic

Maan lo `arr = [10, 20, 30, 40, 50]` aur window size $K = 3$ hai.

Phase 1: Pehli Window Taiyar Karo (K size tak)

Hum ek loop chalayenge 0 se $K-1$ tak aur elements ko queue me push karte jayenge.

- $i = 0$: Queue me gaya `10`. `q = [10]`
- $i = 1$: Queue me gaya `20`. `q = [10, 20]`
- $i = 2$: Queue me gaya `30`. `q = [10, 20, 30]` Ab hamari pehli window taiyar hai! Hum is queue ke saare elements ko print ya store kar lenge.

Phase 2: Window Ko Slide Karo ($i = K$ se $N-1$ tak)

Ab hum baaki bache elements par challenge aur har naye element par do jhatke marenge:

1. **Purane element ko nikaalo ($q.pop()$):** Window aage badh gayi hai, toh queue ka sabse aage wala element ($q.front()$) ab window se bahar ho gaya. Use pop kar do.
2. **Naye element ko daalo ($q.push(arr[i])$):** Naya incoming element queue ke piche se ghus jayega.

Chalo iska live khel dekhte hain:

- $i = 3$ (Element 40):
- $q.pop()$ kiya $\rightarrow 10$ nikal gaya. $q = [20, 30]$
- $q.push(40)$ kiya $\rightarrow 40$ piche jud gaya. $q = [20, 30, 40] \rightarrow$ **Window 2 Taiyar!**
- $i = 4$ (Element 50):
- $q.pop()$ kiya $\rightarrow 20$ nikal gaya. $q = [30, 40]$
- $q.push(50)$ kiya $\rightarrow 50$ piche jud gaya. $q = [30, 40, 50] \rightarrow$ **Window 3 Taiyar!**

2. Queue Waala C++ Code ($O(N)$ Window Tracking)

Interview me jab tumhe 2D vector me saari windows store karke deni hon, toh queue se aise code likhna:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Solution {
public:
    vector<vector<int>> getAllWindowsWithQueue(vector<int>& arr, int k) {
        int n = arr.size();
        vector<vector<int>> allWindows;
        queue<int> q;

        // Step 1: Pehli window ke K elements ko queue me bhar lo
        for (int i = 0; i < k; i++) {
            q.push(arr[i]);
        }

        // Helper function: Queue ke current state ko ek vector me copy karne ke
        // liye
        auto copyQueueToVector = [](queue<int> tempQ) {
            vector<int> currentWindow;
            while (!tempQ.empty()) {
                currentWindow.push_back(tempQ.front());
                tempQ.pop();
            }
        };
```

```

        return currentWindow;
    };

    // Pehli window ko list me daal do
    allWindows.push_back(copyQueueToVector(q));

    // Step 2: Window ko slide karo i = K se lekar n-1 tak
    for (int i = k; i < n; i++) {
        // 1. Outgoing element ko aage se hatao (FIFO Rule 🔥)
        q.pop();

        // 2. Incoming naye element ko piche se daalo
        q.push(arr[i]);

        // Updated window ko final list me daal do
        allWindows.push_back(copyQueueToVector(q));
    }

    return allWindows;
}

};

int main() {
    vector<int> arr = {10, 20, 30, 40, 50};
    int k = 3;

    Solution sol;
    vector<vector<int>> ans = sol.getAllWindowsWithQueue(arr, k);

    // Final matrix print karne ke liye
    for (int i = 0; i < ans.size(); i++) {
        cout << "Window " << i + 1 << " -> [ ";
        for (int x : ans[i]) {
            cout << x << " ";
        }
        cout << "]" << endl;
    }

    return 0;
}

```

3. Complexity Analysis (Queue Ke Saath)

- **Window State Time Complexity:** $O(N)$ 🔥
- **Kyun?** Pure main loop me hum array ke har element par sirf ek baar ja rahe hain. Har element queue me strictly **ek baar push** hota hai aur strictly **ek baar pop** hota hai. Dono operations $O(1)$ hote hain, isiliye window ka structural movement strictly linear ($O(N)$) hai.

- *Note:* Agar hum har window par queue ko copy/print karne ka time bhi jodein, toh copy karne me K operations lagte hain, jisse overall copy time $O(N \times K)$ jata hai (jo ki mandatory hai kyunki output ka size hi utna bada hai).
- **Space Complexity: $O(K)$ Auxiliary Space** 🔥
- **Kyun?** Kyunki kisi bhi ek point of time par queue ka size K se bada nahi ho sakta. Hum hamesha ek element nikalte hain aur ek daalte hain, jisse space hamesha strictly control me rehti hai.

4. Edge Cases Par Iska Performance

1. Window Size Array Ke Barabar ($K = N$):

- Pehla loop 0 se $N-1$ tak chalega, poora array queue me bhar jayega. Sliding wala second loop ($i = k$ yani $n < n$) condition fail karke bypass ho jayega. Safely ek window return ho jayegi. **(PASS ✓)**

2. Window Size Exactly 1 ($K = 1$):

- Queue me pehle `arr[0]` jayega. Agle step par wo pop ho jayega aur `arr[1]` push ho jayega. Queue ka size hamesha strictly 1 maintain rahega aur har single element sahi se separate ho jayega. **(PASS ✓)**

3. Duplicate aur Zero Elements:

- Queue ko values se koi lena-dena nahi hota, wo FIFO sequence follow karti hai. Isiliye duplicates aane par bhi sabse pehle aaya hua element hi bahar niklega, order me zero error aayega. **(PASS ✓)**

Bhai, ab samajh aaya ki Queue ka FIFO nature kaise sliding window ke linear shift ke sath ekdum hand-in-hand match karta hai? Ab dono tarike (index math aur queue state) ekdum solid clear hain?

Bhai, tumne ekdum kantaap pakada hai! Interviewer aksar yeh shart rakh deta hai: **"Bhai, Queue ya Deque jaisa koi extra data structure use nahi karna hai, aur time complexity ko bina nested loop ke optimal rakhna hai."**

Bilkul kar sakte hain! Jab hum queue ya extra spaces use nahi kar sakte, toh hum use karte hain **Smart Array Indexing (Pointer Math)**. Isme hum bina kisi extra data structure ke, sirf loops aur indices ke khel se har window ke elements ko perfectly process kar lete hain.

Chalo bina kisi extra space ($O(1)$ Auxiliary Space) ke iska ekdum makkhan **Optimal Approach** dekhte hain.

1. Optimal Logic: Pure Index Math ($O(1)$ Space)

Jab hume koi extra data structure use nahi karna, toh hum array ke indices ka relation nikalte hain.

Agar hamare paas total N elements hain aur window size K hai, toh total number of windows kitni banengi? $N - K + 1$.

Hum ek single main loop chalayenge jo har window ke **Starting Point (Window Start)** ko track karega, aur uske andar ek fixed boundary loop chalayenge jo bina kisi state-shift ke direct elements ko pick karega.

Khel Kaise Chalta Hai?

- **Window 1:** Shuru hogi index 0 se, khatam hogi $0 + K - 1$ par.
- **Window 2:** Shuru hogi index 1 se, khatam hogi $1 + K - 1$ par.
- **General Window i :** Shuru hogi index i se, khatam hogi $i + K - 1$ par.

Hum direct result vector me har window ke elements ko block-by-block push karte chale jayenge bina kisi Queue ke jhanjhat ke.

2. No-Queue Optimal C++ Code ($O(1)$ Extra Space)

Interview me jab interviewer Queue par ban lagaye, toh ye clean code unke saamne rakhna:

```
#include <iostream>
#include <vector>
using namespace std;

class Solution {
public:
    vector<vector<int>> printAllWindowsNoQueue(vector<int>& arr, int k) {
        int n = arr.size();
        vector<vector<int>> allWindows; // Yeh toh answer return karne ke liye
        mandatory hai

        // Total windows banti hain: (n - k + 1)
        // i track kar raha hai har window ka "STARTING INDEX"
        for (int i = 0; i <= n - k; i++) {

            vector<int> currentWindow;

            // Yeh loop strictly current window ke K elements ko uthayega
            // i se shuru hoke (i + k) tak chalega
            for (int j = i; j < i + k; j++) {
                currentWindow.push_back(arr[j]);
            }

            // Poori window taiyar hote hi final list me daal do
            allWindows.push_back(currentWindow);
        }

        return allWindows;
    }
};

int main() {
    vector<int> arr = {10, 20, 30, 40, 50};
    int k = 3;

    Solution sol;
    vector<vector<int>> ans = sol.printAllWindowsNoQueue(arr, k);

    // Result display karne ke liye
    for (int i = 0; i < ans.size(); i++) {
```

```

        cout << "Window " << i + 1 << " -> [ ";
        for (int x : ans[i]) {
            cout << x << " ";
        }
        cout << "]" << endl;
    }

    return 0;
}

```

3. Complexity Analysis (Queue Ke Bina)

- **Time Complexity:** $O(N \times K)$
- **Kyun?** Kyunki hume har window ke saare elements ko output structure me copy karna hi padega. Agar total windows $(N - K)$ hain aur har window me K elements hain, toh copy karne me $O(N \times K)$ time lagna hi lagna hai. Yeh theoretical minimum hai kyunki output ka size hi itna bada hai.
- **Space Complexity:** $O(1)$ **Auxiliary Space** 🔥 🔥 🔥
- **Kyun?** Dhyan se dekho! Humne pure code me koi `queue`, `deque`, `map` ya koi extra array use nahi kiya window ke state ko manipulate karne ke liye. Hum strictly input array ke indices se direct utha kar data fill kar rahe hain. (Final 2D vector `allWindows` space complexity me count nahi hota kyunki wo question ki demand hai return karne ke liye).

4. Saare Edge Cases Ka Dry-Run

Queue nahi hai, toh boundary cases check karna aur bhi zaroori ho jata hai:

Case 1: Max Window Size ($K = N$)

- **Situation:** `arr = [10, 20, 30]`, $K = 3$. Yahan $N = 3$.
- **Loop Calculation:** Outer loop chalega $i \leq n - k$ yaani $i \leq 3 - 3 \rightarrow i \leq 0$. Yani outer loop sirf **ek baar** chalega ($i = 0$).
- Inner loop $j = 0$ se $j < 3$ tak chalega aur `[10, 20, 30]` ko ek hi window me daal dega.
- **Result:** Strictly single window output. Ek dum **PASS** ✅

Case 2: Min Window Size ($K = 1$)

- **Situation:** `arr = [5, 10]`, $K = 1$. Yahan $N = 2$.
- **Loop Calculation:** Outer loop chalega $i \leq 2 - 1$ yaani $i \leq 1$. Loop chalega $i = 0$ aur $i = 1$ ke liye.
- For $i = 0$: Inner loop $j = 0$ se $j < 1$ chalega. Window 1 = `[5]`.
- For $i = 1$: Inner loop $j = 1$ se $j < 2$ chalega. Window 2 = `[10]`.

- **Result:** Har element apni akeli frame me safely bina kisi crash ke split ho gaya. Ek dum **PASS** ✓

Case 3: Zeros, Negatives, and Duplicates

- Chunki hum sirf loop control variable (**i** aur **j**) par depend kar rahe hain aur elements ke andar ke data ko check hi nahi kar rahe (koi **if** condition nahi hai), isliye array me duplicate elements hon ya negative numbers, memory ya sequence par koi asar nahi padega. Ek dum **PASS** ✓

Bhai, ab samajh aaya ki bina Queue ke bhi sirf direct array pointer aur index manipulation se space ko kaise ek dum $O(1)$ kiya jata hai?

Bhai, is Queue-based Sliding Window approach par interviewer jab edge cases fekta hai, toh wo yeh dekhna chahta hai ki tumhara queue pointer aur bounds calculation mathematically coordinate kar rahe hain ya nahi.

Chalo, is Queue approach ke **saare critical edge cases** ka poora breakdown dekhte hain taaki tumhara code har situation me flawless chale.

1. Window Size Jab Array Size Ke Barabar Ho ($K = N$)

Situation: Maan lo `arr = [10, 20, 30]` aur window size $K = 3$ hai ($N = 3$).

Dry Run:

- **Step 1 (Pehli Window):** Pehla loop **i** = 0 se 2 tak chalega aur saare elements queue me bhar dega.
- `q = [10, 20, 30]`
- Yeh pehli window perfectly `allWindows` vector me push ho jayegi.
- **Step 2 (Sliding Window):** Agla loop **i** = **k** yaani **i** = 3 se shuru hoga. Loop ki condition hai **i** < **n** yaani **3** < **3**, jo ki **false** hai!
- **Result:** Sliding loop chalega hi nahi, aur bina kisi out-of-bound memory error ke safely strictly ek hi window return hogi: `[10, 20, 30]`. (PASS ✓)

2. Window Size Jab Ek dum Minimum Ho ($K = 1$)

Situation: Maan lo `arr = [10, 20, 30]` aur window size $K = 1$ hai.

Dry Run:

- **Step 1 (Pehli Window):** **i** = 0 ke liye loop chalega. Queue me sirf ek element jayega: `q = [10]`. Yeh pehli window store ho gayi.
- **Step 2 (Sliding Window):** Loop **i** = 1 se 2 tak chalega:
- **i = 1 (Element 20):** `q.pop()` se 10 nikla $\rightarrow$ `q = []`. Fir `q.push(20)` hua $\rightarrow$ `q = [20]`. Window 2 store ho gayi.

- **i = 2 (Element 30):** `q.pop()` se 20 nikla $\rightarrow q = []$. Fir `q.push(30)` hua $\rightarrow q = [30]$. Window 3 store ho gayi.
- **Result:** Queue ka size har ek step par strictly 1 maintain raha. Koi empty queue check fail nahi hua, aur saare elements apni akeli-akeli window banakar perfectly return ho gaye. (PASS )

3. Duplicate aur Zero Elements Ka Crossover

Situation: Array me lagatar same values hain, jaise `arr = [5, 5, 5]`, $K = 2$.

Trap: Kuch log sote hain ki duplicate value aane par `pop` galat element ko hata dega.

- **Reality:** Queue elements ki values ko dekhkar kaam nahi karta, wo hamesha positions (FIFO order) par chalta hai.
- Jab Window 1 se Window 2 par slide hoga, toh queue ka `.pop()` aankh band karke index 0 wale 5 ko hi nikalega, chahe index 1 par bhi 5 hi kyun na baitha ho. Isliye value duplicates hone par order me 0% error ka chance hota hai. (PASS )

4. Negative Numbers Waala Flow

Situation: Array me mixed signs hain, jaise `arr = [-1, 0, -2]`, $K = 2$.

Reality: Chunki is question me hume koi condition check nahi karni (< 0 ya > 0), hume bas elements ko capture karna hai, isliye numbers ke signs badalne se queue ke push aur pop mechanism par rati bhar bhi farq nahi padta. Data stream ekdum smoothly flow karegi. (PASS )

Interview Cheat-Sheet: Edge Cases Verification

Edge Case Scenario	Potential Danger	Why Queue Logic Wins?	Result
$K = N$ (Max Window)	Infinite Loop / Index Error	Sliding loop condition $i < n$ stops the execution instantly before any pop.	PASS 
$K = 1$ (Min Window)	Queue Underflow during Pop	Every iteration pops exactly 1 element and pushes 1 element, keeping size strictly at 1.	PASS 
Duplicate Elements	Wrong Element Removal / State Corrupt	Queue follows strict memory sequence (FIFO), bypassing value checking completely.	PASS 
Negative / Zeros	Evaluation Failure	No structural condition depends on array value; pure linear indexing flow.	PASS 

Bhai, ab Sliding Window ke dono core mechanics—**Index Pointer Math ($O(1)$ Space)** aur **Queue State Tracking ($O(K)$ Space)**—unke saare brute, better, optimal, aur micro edge cases ke sath tumhare dimaag me chhap chuke hain!

Ab is topic me interviewer tumhe ek percent bhi nahi fasa sakta. Kuch aur twist dekhna hai isme, ya aage badhein?

First Non Repeating Char in a stream of Character

Bhai, "**First Non-Repeating Character in a Stream**" ka **Brute Force Approach** ekdum detail me samajhte hain. Jab interviewer tumse bole ki "*Mujhe koi fancy data structure (jaise Queue ya Map) nahi dekhna, tum ekdum basic loops se shuruat karo,*" tab tum ye approach unke samne rakhoge.

Brute force ka basic funda ye hai: "**Jaise-jaise stream se naya character aaye, use ek string me jodte jao (record maintain karne ke liye). Aur har naye character ke aane par, shuruat se lekar aakhiri tak dobara loop chalakar check karo ki kaun sa character pehli baar unique dikh raha hai.**"

1. Brute Force Ka Algorithm aur Step-by-Step Logic

Maan lo hamare paas ek khaali string `tempStr = ""` hai jisme hum stream ke characters ko ek-ek karke store karenge, aur ek final `result = ""` string banayenge answer track karne ke liye.

Jab bhi stream se koi character `ch` aayega:

1. **Step 1:** Naye character `ch` ko `tempStr` ke aakhiri me append (add) kar do.
 2. **Step 2 (The Reset Search):** Ek boolean variable banao `foundUnique = false`. Ab hum `tempStr` par **shuruat se (index `j = 0`)** ek scan shuru karenge.
 3. **Step 3 (Frequency Count):** `j` waale character ke liye, hum poori `tempStr` me uska count dhoondenge (ki wo kitni baar aaya hai):
 - Hum ek naya loop chalayenge jo check karega ki poorie `tempStr` me us character ki frequency kya hai.
 - **Agar frequency exactly 1 milti hai:** Iska matlab hume hamara sabse pehla non-repeating character mil gaya! Hum `foundUnique = true` karenge, us character ko `result` me add karenge, aur **break** karke loop se bahar aa jayenge (kyunki hume *first* unique chahiye, aage check karne ki zaroorat nahi).
 4. **Step 4 (No Unique Case):** Agar humne poori string scan kar li aur aisa koi character nahi mila jiski frequency `1` ho, toh iska matlab abhi koi bhi unique character nahi bacha hai. Hum `result` me `#` (ya `_`) jod denge.
-

2. Brute Force Ka Detailed Dry-Run

Chalo ek mast stream lekar isko step-by-step todte hain. Maan lo `stream = "aabc"`.

Step 1: `i = 0`, Character aaya '`a`'

- `tempStr` ban gayi: "`a`"
- Humne shuruat se scan kiya. Pehla character hai '`a`'. Poori string me '`a`' ki frequency check ki → count hai `1`.
- Pehla non-repeating char mil gaya: '`a`'.
- **Result so far:** "`a`"

Step 2: `i = 1`, Character aaya dobara '`a`'

- `tempStr` ban gayi: "`aa`"

- Humne fir se index 0 se scan shuru kiya. Index 0 par baitha hai 'a'. Poori string ("aa") me 'a' ki frequency check ki → count ho gaya 2 (Unique nahi hai!).
- Hum index 1 par gaye, wahan bhi 'a' hai, uski frequency bhi 2 hai.
- Poori string scan ho gayi, koi unique nahi mila. Toh humne result me # daal diya.
- **Result so far:** "a#"

Step 3: i = 2, Character aaya 'b'

- tempStr ban gayi: "aab"
- Humne fir se index 0 se scan shuru kiya.
- Index 0 ('a'): Frequency "aab" me 2 hai (Skip ✗).
- Index 1 ('a'): Frequency 2 hai (Skip ✗).
- Index 2 ('b'): Frequency "aab" me exactly 1 hai! (Unique mil gaya! 🎉)
- Hum turant break karke loop se bahar aaye aur 'b' ko result me jod diya.
- **Result so far:** "a#b"

Step 4: i = 3, Character aaya 'c'

- tempStr ban gayi: "aabc"
- Humne fir se index 0 se scan shuru kiya.
- Index 0 aur 1 ('a'): Frequency 2 hai (Skip ✗).
- Index 2 ('b'): Frequency "aabc" me abhi bhi exactly 1 hai! (Unique mil gaya! 🎉)
- Dhyan se dekho, 'c' bhi unique hai, par hume **FIRST** non-repeating chahiye, isliye chunki 'b' pehle aaya tha aur abhi bhi unique hai, toh hamara answer 'b' hi hoga.
- **Final Result:** "a#bb"

3. Brute Force C++ Code ($O(N^3)$ or $O(N^2)$ strictly based on implementation)

Agar hum ekdum pure raw loops ka use karein bina kisi extra count optimization ke, toh code aisa dikhega:

```
#include <iostream>
#include <string>
using namespace std;

class Solution {
public:
    string firstNonRepeatingBrute(string stream) {
        string tempStr = ""; // Humaari chalti-firti custom stream storage
        string result = "";
```

```
// Outer loop: Stream ke ek-ek naye character ko process karne ke liye
for (int i = 0; i < stream.length(); i++) {
    tempStr += stream[i]; // Naya char append kiya

    bool foundUnique = false;

    // Inner Loop 1: tempStr ke shuruat se check karo ki kaun sa pehla
unique hai
    for (int j = 0; j < tempStr.length(); j++) {
        char currentChar = tempStr[j];
        int frequency = 0;

        // Inner Loop 2: Current character ka poori tempStr me count
nikalne ke liye
        for (int k = 0; k < tempStr.length(); k++) {
            if (tempStr[k] == currentChar) {
                frequency++;
            }
        }

        // Agar frequency exactly 1 hai, toh yahi hamara sabse pehla
unique char hai
        if (frequency == 1) {
            result += currentChar;
            foundUnique = true;
            break; // Pehla unique milte hi aage ke characters ko check
karna band karo
        }
    }

    // Agar saare characters check karne ke baad bhi koi unique nahi mila
    if (!foundUnique) {
        result += '#';
    }
}
return result;
};

int main() {
    Solution sol;
    string stream = "aabc";

    string ans = sol.firstNonRepeatingBrute(stream);
    cout << "Brute Force Output: " << ans << endl; // Output: a#bb

    return 0;
}
```

4. Complexity Analysis (Interviewer ka Punch Point)

Interviewer tumse poochega: "Bhai, ye code chal toh perfectly raha hai, par ye ganda kyun hai?"

- **Time Complexity:** $O(N^3)$ **Worst Case me** ❌
- **Kyun?** Ek loop chal raha hai stream ke length ke liye (N). Uske andar dusra loop chal raha hai `tempStr` ko scan karne ke liye (jo badhte-badhte N tak jayega). Aur uske bhi andar ek teesra loop chal raha hai har character ki frequency dobara zero se count karne ke liye (N).
- Toh $N \times N \times N = O(N^3)$ operations tak baat pahunch jati hai. Agar tum isme standard frequency array use karke frequency count ko $O(1)$ kar do, toh bhi ye nested traversal ki vajah se $O(N^2)$ se kam nahi ho sakta.
- **Space Complexity:** $O(N)$ **Auxiliary Space**
- **Kyun?** Kyunki humne stream ke characters ko dynamically store karne ke liye `tempStr` banayi hai, jo worst-case me pure string ke size (N) ke barabar memory legi.

Bhai, brute force ka poora nested khel ek-ek loop ke breakdown ke sath samajh aaya? Isiliye jab hum optimal par jaate hain, toh hum **Queue** ($O(1)$ **front eviction**) aur **Frequency Array** ($O(1)$ **count check**) ka use karke is pure bwaal ko $O(N)$ me wrap up kar dete hain!

Bhai, ab brute force ke baad aate hain iske **Better Approach** par.

Brute force me dikkat kya thi? Hum har naye character ke aane par poori string me piche jaakar characters ki frequency baar-baar zero se count kar rahe the, jisse time complexity $O(N^2)$ ya $O(N^3)$ ja rahi thi.

Better approach ka main aim hai: "**Frequency count karne ka jhanjhat ek jhatke me khatam karo** ($O(1)$ **count check**), **lekin stream me piche mudkar dekhne ka loop abhi bhi thoda optimized chalega.**"

1. Better Logic: Frequency Array (Hash Map) Ka Khel

Hum 26 size ka ek frequency array ya hash map maintain karenge: `count[26] = {0}`. Isme lowercase alphabets ('a' se 'z') ki live frequency save hoti rahengi.

Steps Kaise Kaam Karenge?

1. Jaise hi stream se naya character `ch` aaye, uski frequency array me badha do: `count[ch - 'a']++`.
2. Saath hi saath, us character ko hum apni ek `string processedStream` me jodte challenge taaki hume characters ka aane ka sequence (order) yaad rahe.
3. **The Search Loop:** Ab hume sabse pehla non-repeating character dhoondna hai. Toh hum apni `processedStream` par shuruat se (`j = 0` se) ek loop chalayenge:
 - Hum check karenge: `if (count[processedStream[j] - 'a'] == 1)`.
 - Jaise hi pehla aisa character milega jiska live count strictly `1` hai, wahi hamara answer hoga! Hum use `result` me add karenge aur `break` kar denge.
 - Agar loop poora khatam ho gaya aur koi aisa char nahi mila jiska count `1` ho, toh hum `result` me `#` jod denge.

2. Better C++ Code ($O(N^2)$ Worst Case, Par Direct Count Check)

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Solution {
public:
    string firstNonRepeatingBetter(string stream) {
        int count[26] = {0}; // Frequency Array (Hash Map alternative)
        string processedStream = ""; // Sequence maintain karne ke liye
        string result = "";

        // Loop poorie stream par chalega
        for (int i = 0; i < stream.length(); i++) {
            char ch = stream[i];

            // 1. Live frequency update karo aur sequence list me daalo
            count[ch - 'a']++;
            processedStream += ch;

            bool foundUnique = false;

            // 2. Ab sequence list me shuruat se check karo kis ka count 1 hai
            // Yeh loop brute force se better hai kyunki frequency count O(1) me
            // mil raha hai
            for (int j = 0; j < processedStream.length(); j++) {
                if (count[processedStream[j] - 'a'] == 1) {
                    result += processedStream[j];
                    foundUnique = true;
                    break; // Pehla unique milte hi ruk jao
                }
            }

            // 3. Agar poori list me koi unique nahi bacha
            if (!foundUnique) {
                result += '#';
            }
        }
        return result;
    }
};

int main() {
    Solution sol;
    string stream = "aabc";

    string ans = sol.firstNonRepeatingBetter(stream);
    cout << "Better Approach Output: " << ans << endl; // Output: a#bb

    return 0;
}
```

```
}
```

3. Complexity Analysis (Interviewer Kahan Tokega?)

- **Time Complexity:** * **Best Case:** $O(N)$ → Agar saare characters unique hon (jaise "abcdef"), toh andar wala loop har baar pehle hi index ($j = 0$) par `count == 1` dekh kar turant break ho jayega.
- **Worst Case:** $O(N^2)$ ❌ → Agar stream aisi ho jisme unique characters ekdum aakhiri me hon ya saare duplicate hote ja rahe hon (jaise "aaaaa" ya "abcdefg..." ke baad bwaal badhe), toh andar wala loop har baar badhti hui string `processedStream` par poora chalega.
- **Space Complexity:** $O(N)$
- **Kyun?** Chunki humne stream ke saare characters ko store karne ke liye `processedStream` string banayi hai, jo memory me input size N ke barabar space legi.

4. Interviewer Ka Agla Sawaal (The Transition to Optimal)

Interviewer code dekh kar boleگا: "Bhai, tumne frequency array lagakar inner count loop toh hata diya, jo ki achha hai. Par tum abhi bhi har character par `processedStream` me shuruat se loop chala rahe ho ($j = 0$ se). Agar meri stream me 10^6 characters hain, toh ye $O(N^2)$ worst case fail ho jayega. Kya tum bina is `processedStream` par baar-baar loop chalaye, direct $O(1)$ me nikal sakte ho?"

Aur yahi par entry hoti hai **Optimal Approach** ki!

Optimal me hum is `processedStream` ki jagah **Queue** use karte hain. Queue ka fayda ye hota hai ki jo element duplicate ho chuka hai, use hum front se **pop** karke hamesha ke liye nikal dete hain, jisse hume kabhi bhi piche ke elements par dobara loop nahi chalana padta!

Bhai, Better approach ka frequency mechanism samajh aaya? Ab iska absolute optimal ($O(N)$ Time, $O(1)$ Space) wala Queue wala dimaag dekhein?

Bhai, ab aate hain is question ke **Absolute Optimal Approach** par, jahan hum Time Complexity ko strictly $O(N)$ aur Extra Space ko strictly $O(1)$ kar denge!

Pichli approach (Better Approach) me dikkat ye thi ki hum `processedStream` par baar-baar shuruat se loop chala rahe the. Us loop ko permanent khatam karne ke liye hum use karenge **Queue** aur ek **Frequency Array** ka combo.

1. Optimal Logic: Queue + Frequency Array

Queue ka niyam hota hai **FIFO (First In First Out)**, aur stream me sabse pehle aaya hua non-repeating character dhoondne ke liye FIFO se behtar kuch nahi hai.

Khel Kaise Chalta Hai?

Hum do cheezein maintain karenge:

1. `count[26] = {0}`: Har character ka live count track karne ke liye.
2. `queue<char> q`: Sirf un characters ko store karne ke liye jo unique hone ki race me hain.

Stream se jaise hi naya character `ch` aayega, hum bas 3 steps lenge:

- **Step 1 (Insert):** Uski frequency badhao (`count[ch - 'a']++`) aur use queue me push kar do (`q.push(ch)`).
- **Step 2 (The Clean-up / Eviction):** Ab hum queue ke front (sabse aage) wale character ko dekhenge. Agar wo character ab duplicate ho chuka hai (yaani uska `count > 1` ho gaya hai), toh wo kabhi hamara answer nahi ban sakta. Use queue se nikaal pheko (`q.pop()`). Yeh tab tak karo jab tak queue khaali na ho jaye ya koi saccha unique character (`count == 1`) front par na mil jaye.
- **Step 3 (Answer):** Agar queue empty ho gayi, toh iska matlab koi unique char nahi bacha, `#` return karo. Agar queue me element hai, toh bina kisi loop ke direct `q.front()` hi hamara answer hai!

2. Optimal C++ Code ($O(N)$ Time, $O(1)$ Space)

```
#include <iostream>
#include <queue>
#include <string>
using namespace std;

class Solution {
public:
    string firstNonRepeatingOptimal(string stream) {
        int count[26] = {0}; // Strictly fixed size array
        queue<char> q;
        string result = "";

        for (int i = 0; i < stream.length(); i++) {
            char ch = stream[i];

            // 1. Frequency badhao aur queue me push karo
            count[ch - 'a']++;
            q.push(ch);

            // 2. Queue ke front se saare bekar/duplicate elements ko saaf karo
            // Yeh loop bohot kam chalega kyunki element ek baar pop hone ke baad
            // dobara nahi aata
            while (!q.empty() && count[q.front() - 'a'] > 1) {
                q.pop();
            }

            // 3. Current stream state ka answer direct O(1) me note karo
            if (q.empty()) {
                result += '#';
            } else {
                result += q.front();
            }
        }

        return result;
    }
};
```

```

    }
};

int main() {
    Solution sol;
    string stream = "aabc";

    string ans = sol.firstNonRepeatingOptimal(stream);
    cout << "Optimal Approach Output: " << ans << endl; // Output: a#bb

    return 0;
}

```

3. Complexity Analysis (Interviewer Selection Point)

Yahan par tumhe interviewer ko asali logic samjhana padega ki **while** loop hone ke baad bhi complexity $O(N)$ kaise hai.

- **Time Complexity:** $O(N)$ Overall 🔥
- **Kyun?** Pure stream processing me, koi bhi character queue me maximum **ek hi baar push** hota hai aur maximum **ek hi baar pop** ho sakta hai. Aisa nahi hai ki har character ke liye while loop poora chalega. Amortized (average) dekhoge toh har character par constant $O(1)$ time hi lagta hai. Isiliye total time strictly linear $O(N)$ hai.
- **Space Complexity:** $O(1)$ Auxiliary Space 🔥 🔥 🔥
- **Kyun?** Humne string size N ke upar depend hone wali **processedStream** ko hata diya hai. Frequency array hamesha fixed size 26 ka rahega. Aur queue me bhi maximum 26 characters hi aa sakte hain (kyunki lowercase alphabets sirf 26 hote hain, usse zyada unique elements bachenge hi nahi). Isiliye extra space absolute constant $O(1)$ hai!

4. Saare Edge Cases Ka Live Dry-Run

Case 1: Saare Characters Same Hon (**stream** = "aaa")

- $i = 0$ ('a'): count=1, $q = [a]$. Front unique hai. Result: a
- $i = 1$ ('a'): count=2, $q = [a, a]$. While loop chalega: front par a hai jiska count 2 hai $\rightarrow$ pop. Agla element bhi a hai $\rightarrow$ pop. Queue khaali ho gayi. Result: #
- $i = 2$ ('a'): count=3, $q = [a]$. Front ka count > 1 hai $\rightarrow$ pop. Queue fir khaali. Result: #
- **Result:** a##. (PASS ✅)

Case 2: Ek Bhi Duplicate Na Ho (**stream** = "abc")

- Har naye char par count hamesha 1 rahega. While loop kabhi condition hit hi nahi karega. Queue me elements append hote jayenge $[a]$, $[a, b]$, $[a, b, c]$. Front par hamesha sabse pehla unique character 'a' hi maintain rahega.

- **Result:** `aaa`. (PASS ✓)

Case 3: Pehla Unique Baad Me Duplicate Ho Jaye (`stream = "abac"`)

- `i = 0 ('a') → q = [a]`, Result: `a`
- `i = 1 ('b') → q = [a, b]`, Result: `a`
- `i = 2 ('a') →` count of `a` becomes 2. `q = [a, b, a]`. While loop front ke `a` ko pop kar dega kyunki uska count > 1 hai. Ab queue bachi `[b, a]`. Front par `b` hai jiska count 1 hai, loop stop. Result: `b`
- `i = 3 ('c') → q = [b, a, c]`. Front par `b` hai (count=1). Result: `b`
- **Result:** `aabb`. (PASS ✓)

Bhai, ab is standard streaming problem ka Brute ($O(N^3)$), Better ($O(N^2)$) aur Optimal ($O(N)$ Time, $O(1)$ Space) ka poora micro-level structure dimaag me lock ho gaya?

Bhai, "**First Non-Repeating Character in a Character Stream**" ke is Optimal Queue Approach par interviewer tumhe phasane ke liye bohot hi bwaal edge cases fek sakta hai. Wo check karega ki jab data continuous aa raha hai, toh kya tumhari queue safely clean ho rahi hai aur boundaries parse ho rahi hain ya nahi.

Chalo, is optimal queue system ke **saare critical edge cases** ka poora dry-run post-mortem dekhte hain.

1. Alternating Duplicates (Trap Case)

Situation: `stream = "abab"` Interviewer ka doubt: "*Bhai, agar character alternate karein, toh kya purana duplicate sahi time par nikal jayega aur naya character system ko track rakhega?*"

Dry Run:

- `i = 0 ('a'):` `count['a'] = 1`, `q = [a]`. Front par `'a'` hai (count=1). Result: `a`
- `i = 1 ('b'):` `count['b'] = 1`, `q = [a, b]`. Front par abhi bhi `'a'` baitha hai jiska count abhi bhi 1 hai, toh while loop nahi chalega. Result: `a`
- `i = 2 ('a'):` `count['a'] = 2` (Duplicate ho gaya!), `q = [a, b, a]`.
- While loop check karega: `q.front()` par `'a'` hai aur `count['a'] > 1` hai. Toh `'a'` pop ho gaya!
- Ab `q = [b, a]`. Front par `'b'` hai aur `count['b'] == 1` hai. Loop ruk gaya. Result: `b`
- `i = 3 ('b'):` `count['b'] = 2`, `q = [b, a, b]`.
- While loop check karega: `q.front()` par `'b'` hai aur `count['b'] > 1` hai → pop! `q = [a, b]`.
- Agla check: `q.front()` par `'a'` hai aur `count['a'] > 1` hai → pop! `q = [b]`.
- Agla check: `q.front()` par `'b'` hai aur `count['b'] > 1` hai → pop! `q = []`.
- Queue empty ho gayi! Result: `#`
- **Result:** `"aabb"`. Stream ka alternate loop ekdum makkhan **PASS** ✓ ho gaya.

2. K-Size Core Stream Me Saare Characters Unique Hon

Situation: `stream = "abcdefg"` (No repetitions at all)

Dry Run:

- Har naye character ke aane par frequency strictly **1** hi rahegi.
 - `while (!q.empty() && count[q.front() - 'a'] > 1)` wala loop **ek baar bhi nahi chalega** (0% performance overhead).
 - Queue me elements line se piche judte jayenge: `[a], [a, b], [a, b, c]`... aur har step par bina kisi loop ke $O(1)$ me hamesha front element `'a'` hi return hota rahega.
 - **Result:** `"aaaaaaa"`. (PASS ✓)
-

3. Ek Hi Character Baar-Baar Aaye (All Same Characters)

Situation: `stream = "aaaa"`

Dry Run:

- `i = 0`: `count['a'] = 1, q = [a]`. Result: `a`
 - `i = 1`: `count['a'] = 2, q = [a, a]`. While loop active hua kyuki front ka count > 1 hai. Dono `'a'` pop ho gaye. `q = []`. Result: `#`
 - `i = 2`: `count['a'] = 3, q = [a]`. Front par jo `'a'` hai uska count 3 hai (yaani > 1). Pop ho gaya. `q = []`. Result: `#`
 - **Result:** `"a###"`. Continuous duplication par queue turant khali ho jati hai aur koi garbage data hold nahi karti. (PASS ✓)
-

4. Pehla Character Bohot Late Repeat Ho (Long Distance Duplicates)

Situation: `stream = "bcda..."` aur fir aakhiri me ek aur `'b'` aa jaye. (`"bcdab"`)

Dry Run:

- Shuruat me `"bcda"` tak saare unique hain, toh front par hamesha `'b'` rahega. `q = [b, c, d, a]`. Result: `"bbbb"`
 - Aakhiri step par jab naya `'b'` aaya: `count['b'] = 2` ho gaya. `q = [b, c, d, a, b]`.
 - While loop check karega: Front par `'b'` hai aur uska count 2 hai → Immediate **Pop!**
 - Ab `q = [c, d, a, b]`. Front par kaun aaya? `'c'`. Kya `'c'` ka count > 1 hai? Nahi (`count['c'] == 1`). Loop waheen ruk gaya!
 - **Result:** Final char mil gaya `'c'`. Khichdi ke bich me se bhi purana safe character aage aa gaya bina kisi error ke. (PASS ✓)
-

5. Memory Limit Check (Interviewer Special)

Situation: Stream me 10^7 characters hain, par characters sirf lowercase alphabets (`'a'` se `'z'`) hi hain.

Trap: Interviewer bolega: *"Bhai stream bohot lambi hai, queue overflow ho jayegi!"*

- **Counter Punch:** Bhai, character sirf 26 hi hain hamare paas. Jab unique elements repeat honge, toh while loop unhe queue se pop karke nikaalta chala jayega. Worst case me bhi queue me maximum **26 elements** hi reh sakte hain (agar saare alphabets ek-ek baar bina repeat hue aaye hon).
- Isiliye stream chahe crore characters ki ho, queue ka size kabhi 26 se upar nahi koodega. Memory absolute safe hai! (PASS )

Interview Cheat-Sheet: Edge Cases Verification

Scenario	Potential Trap	Why Queue Combo Wins?	Result
Saare Same ("aaaa")	Queue me data stack up ho jayega?	While loop front ko turant saaf kar deta hai, max size 1 hi rehta hai.	PASS 
Alternate ("abab")	Order bigad jayega?	FIFO rule naye duplicates ke aane par purano ko safely line-by-line evict karta hai.	PASS 
No Duplicates	While loop slow chalega?	Condition count > 1 hit hi nahi hogi, direct $O(1)$ front pointer read.	PASS 
Bohot Lambi Stream	Out of Memory Error?	Letters limited (26) hain, duplicates pop hote rehte hain, Space strictly $O(1)$ constant.	PASS 

Bhai, ab is Streaming problem ke saare dimensions aur tricky corner cases tumhare dimaag me ekdum solid load ho chuke hain! Kuch aur dhasu question uthayein ab?